

Date de création	07/02/26
Date de dernière mise à jour	11/02/26
Auteur(s)	PIOLET Jean-Marc
Matière (E5-E6-E7-AP)	E5
Compétence validées	

SAUVEGARDE BDD

ET

AUTOMATISATION DE LA SAUVEGARDE

sur *glpi*, *FTP* et *Crontab*

Sommaire

1 – CONNEXION ET SAUVEGARDE.....	3
1.1 <i>MariaDB</i>	3
1.2 <i>Bonne configuration</i>	3
2 – SAUVEGARDE DANS UN SERVEUR DE STOCKAGE.....	4
2.1 <i>Connexion au ftp</i>	4
2.1 <i>Compréhension et transfert du fichier</i>	5
3 – AUTOMATISATION.....	5
3.1 <i>Script</i>	5
3.1 <i>Crontab</i>	6
3.1.A <i>Configurer Crontab</i>	6
3.1.A <i>Execution programmée</i>	7

1 - CONNEXION ET SAUVEGARDE

1.1 MariaDB

On se connecte à notre base de données.

```
root@debian12:~# sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 704
Server version: 10.11.14-MariaDB-0+deb12u2 Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| glpi      |
| information_schema |
| mysql     |
| performance_schema |
| sys       |
+-----+
5 rows in set (0,001 sec)

MariaDB [(none)]> exit
```

On quitte la base de données. On l'extrait vers le dossier */home/debian*, le nom du fichier sera *glpi.sql*

```
MariaDB [(none)]> exit
Bye
root@debian12:~# sudo mysqldump -u root -p glpi > /home/debian/glpi.sql
Enter password:
root@debian12:~#
```

1.2 Bonne configuration

Le fichier se crée bien. Nous aurons besoin de lui mettre un nom de fichier en *Nom_GLPI_AAAAMMDD* le compresser en *tar.gz*, le chiffrer.

On exécute la commande suivante :

```
sudo mysqldump -u root -p glpi > /home/debian/Nom_GLPI_$(date +%Y%m%d).sql
```



Permet de directement configurer la date.



glpi.sql

À chaque fois que vous voulez renommer un fichier lors du processus, on écrit le nouveau nom là où la nouvelle extension du fichier : */home/debian/FICHER-A.ext* */home/debian/Nv_FICHER-A.ext*

On compresse le fichier comme voulu :

```
tar -czf /home/debian/Nom_GLPI_$(date +%Y%m%d).tar.gz
/home/debian/Fichier_créer_précédement.sql
```

Nom_GLPI_20260207.tar.gz

On chiffre le fichier avec OpenSSL :

```
openssl enc -aes-256-cbc -salt -in
/home/debian/Fichier_compresser_précédement.tar.gz -out /home/debian/Nom_GLPI_$(
date +%Y%m%d).tar.gz.enc
```

On entre le mot de passe de chiffrement :

```
enter AES-256-CBC encryption password: abcd
Verifying - enter AES-256-CBC encryption password: abcd
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Nom_GLPI_20260207.tar.gz.enc

Pour pouvoir le déchiffrer, on doit utiliser cette commande et réutiliser le même mot de passe :

```
openssl enc -aes-256-cbc -d -pbkdf2 -in/home/debian/Nom_GLPI_20260207.tar.gz.enc
-out /home/debian/Nom_GLPI_$(date +%Y%m%d).sql.gz
```

```
debian@debian12:~$ rm Nom_GLPI_20260207.tar.gz
debian@debian12:~$ ls
Bureau                               Modèles
Documents                             Musique
glpi-agent-1.13-1_all.deb             Nom_GLPI_20260207.tar.gz.enc
glpi-agent-1.13-linux-installer.pl    PIOLET_glpi_2026-02-05.sql
glpi.sql                              PIOLET_glpi_2026-02-05.tar.xz
glpi.sql.tar.gz                       Public
glpi.tar.xz                           Téléchargements
Images                                Vidéos
debian@debian12:~$
openssl enc -aes-256-cbc -d -pbkdf2 -in /home/debian/Nom_GLPI_20260207.tar.gz.enc -out
/home/debian/Nom_GLPI_$(date +%Y%m%d).sql.gz
enter AES-256-CBC decryption password:
bad decrypt
40171A69307F0000:error:1C800064:Provider routines:ossl_cipher_unpadblock:bad decrypt:..
/providers/implementations/ciphers/ciphercommon_block.c:124:
debian@debian12:~$ ls
Bureau                               Musique
Documents                             Nom_GLPI_20260207.sql.gz
glpi-agent-1.13-1_all.deb             Nom_GLPI_20260207.tar.gz.enc
glpi-agent-1.13-linux-installer.pl    PIOLET_glpi_2026-02-05.sql
glpi.sql                              PIOLET_glpi_2026-02-05.tar.xz
glpi.sql.tar.gz                       Public
glpi.tar.xz                           Téléchargements
Images                                Vidéos
Modèles
```

On peut tester le déchiffrement en supprimant l'ancien fichier.

2 - SAUVEGARDE DANS UN SERVEUR DE STOCKAGE

2.1 Connexion au ftp

Pour pouvoir transférer les fichiers vers le serveur de stockage, on va utiliser le service *FTP*.

```
sudo apt install ftp
```

On se connecte ensuite au serveur de fichier configuré à cet effet. **172.16.80.30**

```
ftp 172.16.80.30
Name : admin@leve
Password : Blaise36!SIO
```

2.1 Compréhension et transfert du fichier

De base, le ftp "observe" l'instance de fichier /dossier *root* de votre VM, si le fichier que vous cherchez a transférer n'es pas dans l'instance observée :

|Exemple /home/debian

Le fichier sera pour lui introuvable. On peut vérifier avec *lcd*.

Local directory now:

On se déplace vers le dossier où se trouve le fichier.

Lcd /home/debian

On se rend avec *cd* dans le dossier où on souhaite déposer le fichier. *Ls* pour observer les fichiers présents dans le dossier.

On tape la commande : *put Nom_du_fichier*

```
ftp> put Nom_GLPI_20260207.tar.gz.enc
local: Nom_GLPI_20260207.tar.gz.enc remote: Nom_GLPI_20260207.tar.gz.enc
229 Entering Extended Passive Mode (|||8164|)
150 Ouverture d'une connexion de données en mode BINARY pour Nom_GLPI_20260207.tar.gz.e
nc
100% |*****| 214 KiB 80.93 MiB/s 00:00 ETA
226 Téléchargement terminé
219136 bytes sent in 00:00 (51.39 MiB/s)
ftp> ls
229 Entering Extended Passive Mode (|||7321|)
150 Ouverture d'une connexion de données en mode ASCII pour file list
-rw-r--r-- 1 adminleve adminleve 219136 Feb 7 14:24 Nom_GLPI_20260207.tar.gz.enc
226 Téléchargement terminé
```

3 - AUTOMATISATION

3.1 Script

Pour automatiser tout ça, on a besoin d'un script qui va exporter notre base de données, la compresser et la chiffrer. Il nous restera plus qu'à transférer ce fichier dans le serveur de sauvegarde FTP.

```
#!/bin/bash
# Configuration
DATE=$(date +%Y%m%d)
BACKUP_DIR="/home/debian/backups"
FILENAME="Nom_GLPI_${DATE}"
DB_NAME="glpi"
DB_USER="root"
DB_PASS="votre_mot_de_passe" # ou laisser vide si utilisation de .my.cnf
RETENTION_DAYS=30 # Garder les sauvegardes pendant 30 jours
# Créer le répertoire de sauvegarde s'il n'existe pas
mkdir -p ${BACKUP_DIR}
# Log
echo "[$(date)] Début de la sauvegarde GLPI" >> ${BACKUP_DIR}/backup.log
# 1. Dump de la base de données
mysqldump -u ${DB_USER} ${DB_NAME} > ${BACKUP_DIR}/${FILENAME}.sql 2>> $
```

Les sauvegardes seront stockées dans
/home/debian/backups/

```
{BACKUP_DIR}/backup.log
if [ $? -eq 0 ]; then
    echo "[$(date)] Dump réussi" >> ${BACKUP_DIR}/backup.log
    # 2. Compression
    gzip ${BACKUP_DIR}/${FILENAME}.sql
    echo "[$(date)] Compression réussie" >> ${BACKUP_DIR}/backup.log
    # 3. Chiffrement
    openssl enc -aes-256-cbc -salt -pbkdf2 -in ${BACKUP_DIR}/${FILENAME}.sql.gz
    -out ${BACKUP_DIR}/${FILENAME}.sql.gz.enc -pass pass:VotreMotDePasseChiffrement
    if [ $? -eq 0 ]; then
        echo "[$(date)] Chiffrement réussi" >> ${BACKUP_DIR}/backup.log
        # 4. Nettoyage du fichier .gz non chiffré
        rm ${BACKUP_DIR}/${FILENAME}.sql.gz
        # 5. Suppression des anciennes sauvegardes
        find ${BACKUP_DIR} -name "Nom_GLPI_*.sql.gz.enc" -mtime +$
{RETENTION_DAYS} -delete
        echo "[$(date)] Nettoyage des anciennes sauvegardes terminé" >> $
{BACKUP_DIR}/backup.log
        echo "[$(date)] Sauvegarde terminée avec succès: ${FILENAME}.sql.gz.enc"
>> ${BACKUP_DIR}/backup.log
    else
        echo "[$(date)] ERREUR lors du chiffrement" >> ${BACKUP_DIR}/backup.log
        exit 1
    fi
else
    echo "[$(date)] ERREUR lors du dump" >> ${BACKUP_DIR}/backup.log
    exit 1
fi
```

On rend le script exécutable :

```
sudo chmod +x /usr/local/bin/backup_glpi.sh
```

3.1 Crontab

Pour planifier l'exécution du script à un moment défini, il faut ajouter une tâche dans le planificateur afin de préciser la période d'exécution.

3.1.A Configurer Crontab

Pour éviter de mettre le mot de passe dans le script, créez d'abord un fichier **.my.cnf** :

```
sudo nano /root/.my.cnf
```

Contenu :

```
[client]
user=root
password=votre_mot_de_passe_mysql
```

On sécurise la configuration :

```
sudo chmod 600 /root/.my.cnf
```

3.1.A Execution programmée

On entre dans la table Cron :

```
sudo crontab -e
```

On y précise la planification que l'on souhaite :

```
# Tous les jours à 2h du matin
0 2 * * * /usr/local/bin/backup_glpi.sh
# Tous les jours à 3h30
30 3 * * * /usr/local/bin/backup_glpi.sh
# Tous les lundis à 1h
0 1 * * 1 /usr/local/bin/backup_glpi.sh
# Toutes les 6 heures
0 */6 * * * /usr/local/bin/backup_glpi.sh
```

Et on vérifie que le service fonctionne bien :

```
# Vérifier le service cron
sudo systemctl status cron
# Voir les logs cron
sudo tail -f /var/log/syslog | grep CRON
# Voir les logs de sauvegarde
tail -f /home/debian/backups/backup.log
```

Et on teste le script manuellement pour vérifier que, s'il est planifié sur une période donnée, par exemple toutes les minutes, la sauvegarde se lance correctement pendant l'intervalle choisi.

```
sudo /usr/local/bin/backup_glpi.sh
```